From Prompt to Vulnerability: Evaluating Security Risks in AI-Generated Proxy Tunnel Code Across Multiple Large Language Models

Contents

Introduction:.....3
Literature Review:.....4
Aim:.....8
Objective:.....8
Research Questions:.....8
Methodology:.....9
Design and Technical Approach:.....9
Model Selection and Data Collection:.....10
Technical Environment:.....10
Static Analysis Tools and Justification:.....11
Penetration Testing Procedure and Justification:.....11
Step-by-Step Research Procedure:.....12
Data Analysis:.....13
Ethical Considerations:.....14
Project Timeline:.....15
References.....18

Introduction:

In professional software development, large language models (LLM) have quickly become popular. Since its release, software development has evolved to the point where anyone can create software with simple instructions, with no prior coding knowledge. Tools like ChatGPT and Claude now let developers turn plain-language descriptions into working code. This leads to tasks that used to take hours take minutes instead. Umama et al. (2025) conducted a systematic review of 58 total studies examining LLM-based code generation from 2020 to 2025. The study found a rapid increase in the use of LLMs in corporations and academic institutions. It has fundamentally changed how software is built more than anything in recent years.

However, there is a serious and unexplored security risk associated with the productivity benefits of LLM-generated code. The code generated by these models may contain flaws that neither the model nor the developer fully understands. Unlike humans, LLMs do not reason about the security of the code that they generate. Rather than relying on algorithms or heuristics, they pattern-match using data from their training set, which includes insecure code snippets. Ambati et al. (2024) found that ChatGPT and Google Bard produced insecure code in approximately 50% of cases across Python, Java, and C; however, ChatGPT produced insecure code 90.6% of the time when generating Java code. Furthermore, Bouzid and Khoury (2025) find substantial variation in ChatGPT's ability to generate secure code across programming languages and vulnerability types. This is not simply a defect in a single instance of an LLM; rather, it is endemic to the underlying structure of all LLMs.

The risk is especially significant in network-facing code, where vulnerabilities in input validation, connection management, and data transmission can make systems directly vulnerable to attacks. Proxy tunnel implementation, the focus of this study, involves all these vulnerability surfaces and represents a realistic, security-critical use case for LLM-generated code. Although it is important, no existing research directly compares ChatGPT and Claude on a network-specific implementation task that uses both static analysis and penetration testing. The iterative nature of real-world LLM use, where developers prompt a model repeatedly until working code is produced,

has not been given much attention. According to Ambati et al. (2024), repeated repairs can make the problem worse instead of improving or fixing it.

This research tackles all of these gaps. It tasks ChatGPT and Claude with generating a proxy tunnel implementation in both Python and Java, using a fully documented, iterative prompting process. Then it uses static analysis and penetration testing to check the security of the output. The entire interaction log from every prompting session is considered primary research data. This allow the study to look how vulnerabilities emerge and evolve over the course of the prompting sequence. The deployment at scale of network code generated by LLMs that has unexamined vulnerabilities represents a growing and immediate threat to the security of real-world software systems.

Literature Review:

The security implications of code generated by large language models have become a major concern in modern software engineering research. As LLM-based code generation tools have become standard parts of professional development workflows, a growing number of scholarly research studies have sought to characterize the nature of and occurrence of common bugs in generated code. Two survey studies set the wider context for this problem. Umama et al. (2025), in a PRISMA-guided systematic analysis of 58 studies published between 2020 and 2025. The study has found numerous factors that are commonly cited as drawbacks of using LLMs for generating software code, such as hallucinations, limited generalization, and security vulnerabilities. In their study on LLM applications in software security, Sheng et al. (2025) confirm that LLMs are increasingly able to identify possible vulnerabilities in software. However, they still have restrictions in their ability to produce software code lacking inherent security issues, and therefore, the issue remains unresolved in this research area. Together, these two studies show that the security of LLM-generated code is still a challenge and that the research community lacks uniform evaluation methods.

Using this broad base as a starting point, multiple focused empirical studies have provided particular details on the degree and nature of vulnerabilities in LLM-generated code across models and contexts. For instance, Ambati et al. (2024) conducted an extensive empirical study at the University of Saskatchewan that compared the outputs of ChatGPT and Google Bard (Python,

Java, and C) across 3854 unique code instances. The result was that both models produced code that would be easy to hack approximately 50% of the time. Furthermore, when ChatGPT-generated code was analyzed by itself, nearly 90.6% of the Java code contained verifiable, exploitable vulnerabilities. Bouzid & Khoury (2025) also undertake an extensive systematic review in ACM Computing Surveys of the two created by ChatGPT for the specific long-term purpose of developing secure software. The review showed that the security performance of ChatGPT code changes depends significantly on the programming language, the type of CWE vulnerability, and the code's length and complexity. The authors also found that improvements in LLM generation have not solved the underlying problem. These results, in combination, establish that ChatGPT consistently generates code that is highly vulnerable. However, both studies mostly focus on ChatGPT and do not compare it to Claude. Therefore, a direct model-to-model comparison is absent from the literature. Licorish et al. (2025) further demonstrate this difference by directly comparing GPT-4-generated Python code to human-authored code across 72 software engineering tasks. The authors used Pylint, Radon, Bandit, and test cases to make comparisons. The comparative analysis indicated that while human-written code had a wider range of security problems, GPT-4 produced more severe outliers. Overall, there is strong evidence from Ambati et al. (2024), Bouzid & Khoury (2025), and Licorish et al. (2025) that supports the argument that code generated by LLMs shows unique and greater levels of vulnerability than code written by humans. This provides a clear empirical basis for research that focuses on LLM output.

Notably absent across all three studies is a thorough analysis of how the prompting process shapes the security outcome. Ambati et al. (2024) made an extremely significant observation that when models were tasked with fixing known security vulnerabilities, they often introduced new security weaknesses. The finding shows that prompting models repeatedly may compound rather than resolve the security concern. This finding is directly related to the research by Tony et al. (2025), which offers empirical evidence that various prompting strategies yield significantly different results across GPT-3, GPT-3.5, and GPT-4. This study underscores the effectiveness of applying iterative methods of recursive critique and adjustment over multiple iteration cycles to consistently close security gaps. This corroborates Ambati et al.'s (2024) assertion that the order of prompting is a security-relevant variable. Together, these two earlier studies indicate that it is not possible to reasonably assess the security of LLM-generated code for the entire life cycle of a code component based on a single snapshot in time. Instead, the iterative interaction between model and

developer must be understood. Nonetheless, no current study treats the complete prompting interaction log as primary research data or studies the emergence and the change of vulnerabilities across successive prompts. Thereby, leaving this aspect of the issue systematically unaddressed.

The risks mentioned across the aforementioned studies are particularly acute when LLMs are prompted to write code that connects to the network. These code weaknesses arise from flaws in how inputs are handled, connection management, and data transmission, which can make the system vulnerable to direct attack. A new study by Dunne et al. (2024), the IEEE QRS study, tested several LLMs, including GPT-3.5 and GPT-4. The models were asked to implement a message-handling function for a networking protocol in an embedded system. In their dynamic fuzz testing, they used AFL++ to dynamically fuzz the program's output. This testing found 14 unique types of security vulnerabilities grouped into a 3-way taxonomy. These were grouped based on input validation failure, buffer error handling, and integer management. These vulnerability classes are the same ones that could be found on the attack surface of proxy tunnel implementations. To compare programming languages, Wang et al. (2024) used the CodeSecEval benchmark to analyze 44 CWE categories of vulnerabilities in Java and Python. The study used an extensive analysis of languages and model-based differences. The study concluded that both GPT-4 and Claude 3 Opus frequently ignored security during code generation. Neither model consistently outperformed the other across all vulnerability categories. This constitutes the most direct existing work to the current study, since it directly includes Claude as a test subject. Wang et al. (2024) conducted only static analysis during testing, which may compromise the accuracy of their findings. Kumakale et al. (2026) take this analysis of language a step further by focusing on vulnerabilities specific to Java rather than a comparative review of both programming languages and, using machine learning classifiers, identify 95% of Java programming language security vulnerabilities. This study shows that Java possesses particular and substantial vulnerability surfaces in LLM-generated code that require targeted scrutiny. To summarize, Dunne et al. (2024), Wang et al. (2024), and Kumakale et al. (2026) found that both networks and Java code are high-risk areas for LLM-vulnerable generated code. It further shows that neither has been subjected to a combined static and dynamic security evaluation in a direct Claude vs. ChatGPT comparison.

Two further sources have identified similar methodologies for evaluating tools and methods for assessing the security of LLM-generated code, endorsing the essential approach for assessing the risks associated with it. According to Krebs and Mazumdar (2025), there were nine

LLMs analyzed using Pylint, Radon, and Bandit across three different use case categories in accordance with the ISO/IEC 5055:2021 (Krebs & Mazumdar, 2025) specification, with assertions made as to whether or not LLM-generated software code could be qualified and evaluated based on static analysis tools. These outcomes confirm that developers can, using static analysis tools, consistently evaluate the security and reproducibility of LLM-generated code over time. However, Krebs and Mazumdar (2025) state that using static analysis alone will not determine whether the vulnerability identified by static analysis tools could be exploited in production. Radanliev (2025) tackles this limitation by developing and testing an automated penetration testing framework that uses OWASP ZAP, Burp, and SonarQube to run dynamic and static software security tests against generative AI applications as outlined in the Springer Series on Applied Intelligence. The study produced over 300 findings of vulnerability to attack; a combination of results found a 70% reduction in the number of high-severity issues through combined testing of a sample of LLM applications processed by both types of security analysis. As found by Krebs and Mazumdar (2025), Radanliev (2025) demonstrates that a full and thorough examination of the security of applications generated by LLMs requires both static and active forms of testing so as to confirm the security vulnerability of the software and to confirm that the vulnerability of the software could be exploited in an actual environment. No studies have previously demonstrated the use of a combination of both forms of security analysis in a comparative study involving network-specific proxy tunneling code generated by ChatGPT and Claude in either Python or Java.

The studies reviewed, taken as a whole, provide a unified and cumulative picture; there is consistent evidence across the research of long-standing and continuing security gaps in LLM-generated code identified by Umama et al. (2025) and Sheng et al. (2025). Ambati et al. (2024), Bouzid and Khoury (2025), and Licorish et al. (2025) have quantified the size of the gaps between models and languages. Collectively, Tony et al. (2025) and Ambati et al. (2024) demonstrated that the prompting process has impacted vulnerability outcomes in ways that previous studies have not examined in depth. Krebs and Mazumdar (2025), together with Radanliev (2025), substantiate and endorse the integrated static and dynamic testing methods employed in the current study. The existing literature lacks a study that directly compares Claude and ChatGPT across network-specific implementations in Python and Java. Consequently, utilize both static analysis and penetration testing, considering the entire sequence of iterative prompting as the primary source of research data. The present study intends to address all of the identified gaps.

Aim:

The aim of this study is to perform a comparative security assessment of ChatGPT and Claude by asking both LLMs to generate proxy tunnel code in Python and Java. Next, both static and active penetration testing will be used to identify code flaws, and the entire process will be documented in detail.

Objective:

- Generate a proxy tunnel using an iterative prompting approach with ChatGPT and Claude in both Python and Java.
- Employ static analysis tools to help detect underlying structural faults that exist in the code generated with ChatGPT and Claude, and classify these according to the Common Weakness Enumeration (CWE) framework.
- Conduct penetration tests to confirm the exploitability of any particular vulnerability discovered using static analysis.
- Assess interaction logs for each prompting session of ChatGPT and Claude to understand how vulnerabilities emerge, persist, or are introduced anew through successive iterations of the prompting process.
- To examine the security profiles of Claude and ChatGPT in both programming languages and discover patterns in their vulnerabilities.

Research Questions:

1. What vulnerabilities are present in proxy tunnel code generated by ChatGPT compared to Claude in Java and Python?
2. What vulnerabilities are identified through penetration testing and static analysis in LLM-generated proxy tunnel code, and how does each method compare?

3. How does the iterative sequence of exchange between the user and the model affect the emergence of security vulnerabilities in the final output?

Methodology:

Design and Technical Approach:

The study will implement a technical experimental research design where two LLMs (ChatGPT and Claude) will be prompted to generate a proxy tunnel. The generated code will then be evaluated for security vulnerabilities in both static analysis and active penetration testing. The selection of experimental design is proper, given that the study intends to isolate and evaluate the security behavior of two distinct models under regulated, reproducible conditions. The technical approach unites two aligned security evaluation methods. The purpose of Static Application Security Testing (SAST) is to identify the structural vulnerabilities in the code without executing the application. Dynamic Application Security Testing (DAST) uses automated penetration tests to find vulnerabilities in an application based on how users will interact with it post-deployment. Radanliev (2025) demonstrates that combining SAST and DAST yields more vulnerabilities than either method alone. In contrast, Wang et al. (2024) used only static analysis in their CodeSecEval study and acknowledged that they could not confirm whether the vulnerabilities could be exploited. By using both methods, one can overcome a common issue in traditional studies that rely solely on static analyses of prompts.

The rationale for choosing a proxy as a target implementation is that it is an application available on the network and includes connection management, data forwarding, input validation, and socket management. Dunne et al. (2024) show that input validation, buffer handling, and integer management are the most common points of failure in networking code generated by LLMs. The selection of Python and Java as two target programming languages is due to their broad adoption and extensive research coverage. According to Ambati et al. (2024) and Wang et al. (2024), both Python and Java are extensively used in real-world network application development. This allows for a direct cross-language comparison of vulnerability patterns within the same model.

Model Selection and Data Collection:

The study looks at two models: ChatGPT (GPT-4) and Claude (3.5 Sonnet). ChatGPT was selected for its widely researched LLM in the code security literature. Both Ambati et al. (2024) and Bouzid and Khoury (2025) provided baseline data on its security behavior. Unlike other large commercial language models (LLMs), Claude is often used in software development but receives much less focused security research attention than other models. Wang et al. (2024) include Claude 3 Opus as a subject in CodeSecEval but place little importance on it. Furthermore, no current study directly contrasts Claude with ChatGPT regarding a network implementation task. This gap is what led to the choice of models in this study.

This study uses generated data rather than an existing dataset. Through a structured iterative prompting process, each model will be prompted to generate a functional proxy tunnel implementation in both Python and Java. There will be four prompting sessions: ChatGPT Java, ChatGPT Python, Claude Java, and Claude Python. Every session will start with the same prompt that explains what the proxy tunnel needs to do. The session will then go on through a back-and-forth exchange until the model generates functional code. For each session, all prompts, model responses, error messages, and requests for revision will be recorded in full. This interaction log is the primary source of research data for answering RQ3 and constitutes a methodological addition not found in existing studies. The prompting sessions will be conducted via the standard web interfaces for ChatGPT and Claude, and the results of every exchange will be manually copied from these systems into a single structured log file. Ambati et al. (2024) found that using distinct prompting methods could lead to many more vulnerabilities with each additional prompt. Tony et al. (2025) also prove that prompting methods considerably affect the security risk of the resulting end product.

Technical Environment:

The code will execute on an Ubuntu 22.04 LTS virtual machine, isolated from other VMs. This prevents results from penetration tests from affecting other systems outside the VM, and it allows for repeatable, controlled security testing. Throughout the test, the VM will have no internet access to block network exposure. The testing environment consists of JDK 17 for Java code and Python 3.11 for Python code. Also, an isolated network interface for sending and receiving test data.

Static Analysis Tools and Justification:

There will be two tools for each language to examine the generated code. The first tool is Bandit and will be used for Python security testing. Bandit analyzes code for vulnerabilities and compares the patterns versus the known vulnerabilities. Results will be classified by CWE ID and will indicate the threat level associated with each finding. Bandit was chosen because it was developed specifically for security analysis and provides an easy way to compare it with previously published research.

Krebs and Mazumdar (2025) and Licorish et al. (2025), in their evaluations of LLM-generated code, confirmed that the mentioned tools are valid security analysis tools. SpotBugs with the FindSecBugs plugin will be used for Java analysis. SpotBugs is a static analysis tool for Java that identifies bug patterns through analyzing bytecode. The FindSecBugs plugin adds vulnerability detection, mapped to CWE and OWASP categories. SpotBugs was selected because it is a prominent open-source static security analysis tool for Java and produces output classified by CWE. The output will make it easy to compare languages directly.

A shared limitation of both tools is that static analysis can produce false positives. This is precisely why penetration testing is a useful complementary method. Results from static analysis will be considered signs of possible vulnerability. However, it will be confirmed through dynamic testing.

Penetration Testing Procedure and Justification:

Active penetration testing will be conducted on each proxy tunnel implementation generated by the LLM and run in isolated VM environments. During the active phase of penetration testing, OWASP ZAP will be the primary dynamic testing tool. OWASP ZAP is an open-source tool that allows us to verify the security of web applications by detecting vulnerabilities in running applications that attackers could exploit. The vulnerability will allow attackers to access data via an insecure communication channel, log into a user account, and launch an attack by injecting code into a running application. This tool has been selected for this function because it has been tested and proven effective for testing web applications. Moreover, OWASP ZAP is widely used in academic security research and is one of the tools Radanliev (2025) uses in the penetration testing suite that supports this methodology.

Active penetration testing will be conducted for each implementation and will follow the specified steps. The first step is to install and run the proxy tunnel code in the isolated VM environment. Second, Nmap will be used to identify the attack surface and verify that the tunnel is working by listing all open ports and services. Third, OWASP ZAP will automatically look for common vulnerabilities in the proxy tunnel. Fourth, manual testing will be conducted to determine whether the static analysis phase identified any real vulnerabilities that can be matched to the static findings. Finally, the results will be noted with a severity score per the CVSS framework. CVSS scoring is appropriate because it delivers a standardized way to measure the severity of a vulnerability. This makes it easy to compare different testing methods, languages, and models.

The drawback of penetration testing in this scenario is the potential inability to detect all code vulnerabilities. Particularly those necessitating specific runtime conditions or intricate attack sequences to activate. The results of penetration testing will be used to confirm the findings of the static analysis, rather than providing a separate list of vulnerabilities.

Step-by-Step Research Procedure:

1. Develop a standard base case starting prompt detailing the proxy tunnel requirements to be utilized in each of the four prompt sessions.

2. Conduct four iterative prompting sessions and document everything that happens during the prompting sessions.

3. Capture final working code output from each session for analysis.

4. Use Bandit to test Python output, and use SpotBugs with FindSecBugs to test Java output. Capture results and document findings, as well as severity levels and CWE IDs.

5. For the generated code sample, create separate VMs for testing also to run Nmap and OWASP ZAP.

6. Perform penetration testing based on findings identified in the static testing.

7. Document results from static/dynamic testing, and compare sets for similarities and vulnerabilities.

8. Keep track of the interaction logs from each prompting session to see when vulnerabilities were added or changed between iterations.

9. Compile results into a comparative dataset for comparison, organized by model, language, tool, CWE category, and CVSS severity score.

Data Analysis:

This study employs three data types. For static analysis outcomes from Bandit and SpotBugs. Penetration testing results from OWASP ZAP and interaction logs from each prompting session. Every tool is used differently, and its use is justified in light of the research question.

To address RQ1, which examines differences regarding vulnerability between ChatGPT- and Claude-generated code. A comparative quantitative analysis will be conducted. Through the use of static code analysis and penetration testing, vulnerabilities will be identified and assigned CVSS severity scores along with appropriate CWE categorization. This enables us to directly compare the numbers, types, and severities of vulnerabilities across various languages and models. Additionally, descriptive statistics will be conducted to find the number of vulnerabilities, the severity distributions, and the CWE category frequencies for each of the four code outputs. The quantitative methodology used is appropriate as the research question requires a direct comparison of distinct, classifiable security findings across designated categories. The method described here is similar to the CWE-based classification method used by Wang et al. (2024) in CodeSecEval to compare their model effectiveness with other models using the same methodology.

To answer RQ2, which examines the differences between static analysis and penetration testing, we will examine the results from both methods and group them into three categories. First, vulnerabilities found only using static analysis; second, vulnerabilities found only through penetration testing; and finally, vulnerabilities found by both methods. This approach of cross-referencing will allow for evaluating how much information dynamic testing will provide in comparison to static analysis. This method tackles the gap identified by Wang et al. (2024), namely that static analysis cannot confirm exploitability; however, the confirmation rate from static analysis findings relative to those found during penetration testing will be expressed as the percentage of findings confirmed.

To answer RQ3, this study will examine the interaction logs of each of the four prompting sessions to determine how iterative prompting affects the emergence of weaknesses. Each iteration of the prompting exchange will be analyzed to determine when certain vulnerabilities appear in the code, whether they continue across later iterations, and whether any new vulnerabilities are introduced into the model. This analysis is based on the research of Ambati et al. (2024), who found that successive prompting would make code less secure. Studies by Tony et al. (2025) also found that some prompting strategies could alter security outcomes.

The qualitative analysis will use a deductive coding framework gathered from the vulnerability categories identified in the dynamic and static testing. This will allow us to track how different types of vulnerabilities emerge and evolve throughout the prompting sequence. This approach is valid because RQ3 focuses on a process-oriented assessment of changes regarding vulnerability over time, requiring an analysis of sequential data rather than just totals.

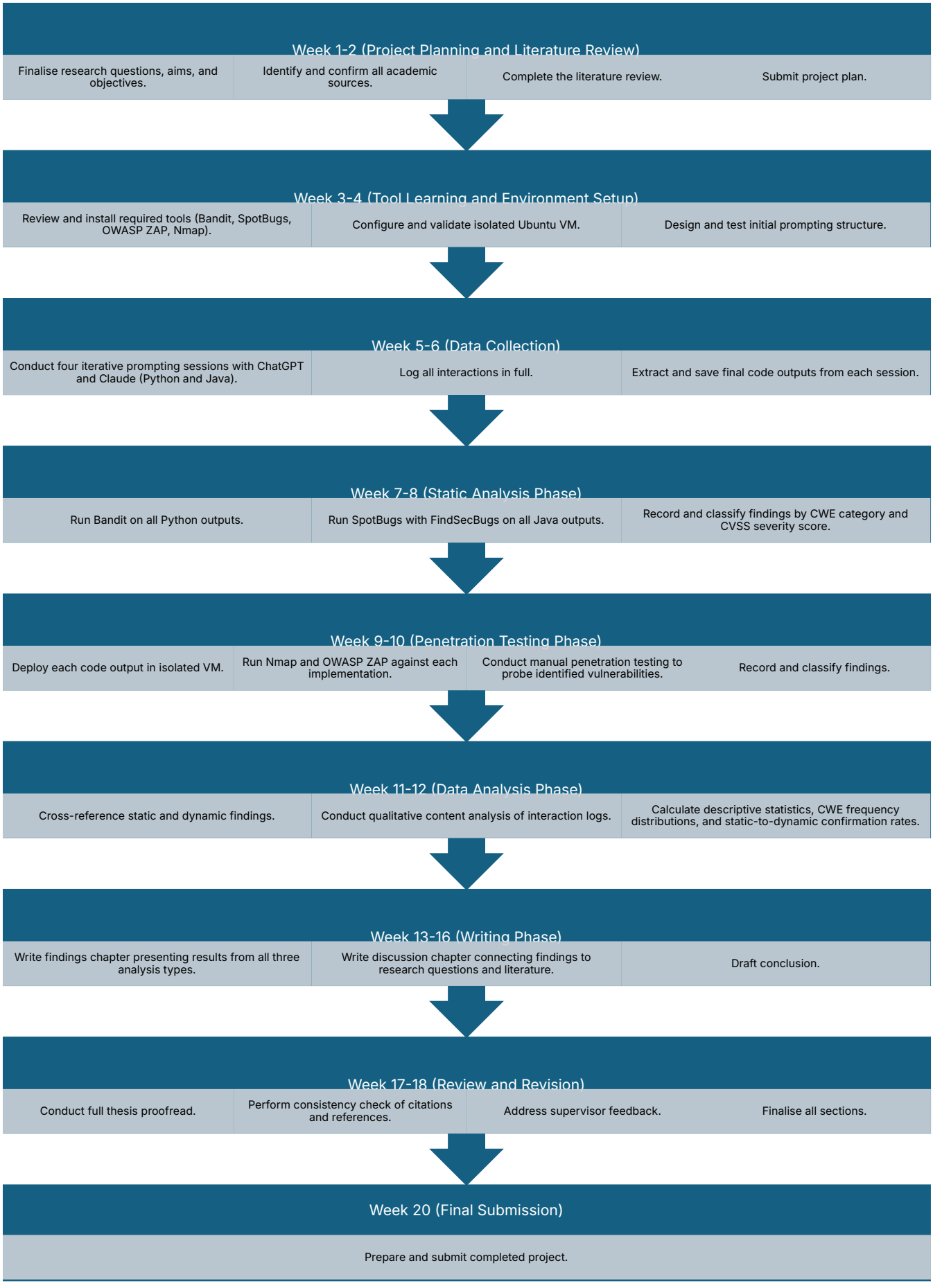
To validate results through triangulation, the findings from static (code) analysis, penetration testing, and interaction logging will be compared. A high level of confidence can be achieved if all three sources conclude the same vulnerabilities. When a difference among methods is found, the discrepancy will be acknowledged and examined as a distinct methodological observation.

Ethical Considerations:

Despite the absence of human participants in this study, it raises some ethical considerations. All penetration testing will only be conducted on an isolated virtual machine environment, ensuring that no external systems are affected and full compliance with computer misuse legislation. During the study, all data, including iterative interaction logs and code samples, will be stored and treated as sensitive research material. If a new pattern of vulnerability emerges, responsible disclosure practices will require informing Anthropic and OpenAI before the information is published publicly. To reduce dual-use risks, this study will present findings at an appropriate abstraction level. This will support defensive practice without constituting a practical attack guide and will not include exploit code. The research results that explain how prompting affects vulnerable coding could be used by malicious users. Therefore, results will be provided in a manner that supports defensive security, with no step-by-step instructions for conducting an attack.

Project Timeline:

The project timeline outlines the planned schedule for finishing each part of the research over the course of five months. The timeline is designed to be realistic, taking into account how complicated the methodology is from a technical perspective.


Week 1-2 (Project Planning and Literature Review)
Finalise research questions, aims, and objectives.
Identify and confirm all academic sources.
Complete the literature review.
Submit project plan.
Week 3-4 (Tool Learning and Environment Setup)
Review and install required tools (Bandit, SpotBugs, OWASP ZAP, Nmap).
Configure and validate isolated Ubuntu VM.
Design and test initial prompting structure.
Week 5-6 (Data Collection)
Conduct four iterative prompting sessions with ChatGPT and Claude (Python and Java).
Log all interactions in full.
Extract and save final code outputs from each session.
Week 7-8 (Static Analysis Phase)
Run Bandit on all Python outputs.
Run SpotBugs with FindSecBugs on all Java outputs.
Record and classify findings by CWE category and CVSS severity score.
Week 9-10 (Penetration Testing Phase)
Deploy each code output in isolated VM.
Run Nmap and OWASP ZAP against each implementation.
Conduct manual penetration testing to probe identified vulnerabilities.
Record and classify findings.
Week 11-12 (Data Analysis Phase)
Cross-reference static and dynamic findings.
Conduct qualitative content analysis of interaction logs.
Calculate descriptive statistics, CWE frequency distributions, and static-to-dynamic confirmation rates.
Week 13-16 (Writing Phase)
Write findings chapter presenting results from all three analysis types.
Write discussion chapter connecting findings to research questions and literature.
Draft conclusion.
Week 17-18 (Review and Revision)
Conduct full thesis proofread.
Perform consistency check of citations and references.
Address supervisor feedback.
Finalise all sections.
Week 20 (Final Submission)
Prepare and submit completed project.

The timeline is subject to change based on the complexity of the generated code and its tests. The iterative prompting phase poses the most significant threat to the project schedule. Given the unpredictability of exchanges required for code functionality. To minimize the risk, a 2-week buffer time has been built into the schedule between the data collection and analysis phases.

References

Ambati, S., Ridley, N., Branca, E., & Stakhanova, N. (2024). Navigating (in)security of AI-generated code. *IEEE International Conference on Cyber Security and Resilience (CSR)*, 30-37. Retrieved from https://ieeexplore.ieee.org/document/10679468

Bouzid, R., & Khoury, R. (2025). Assessing the Effectiveness of ChatGPT in Secure Code Development: A Systematic Literature Review. *ACM Computing Surveys, 57*(12). Retrieved from https://doi.org/10.1145/3744553

Dunne, M., Schram, K., & Fischmeister, S. (2024). Weaknesses in LLM-Generated Code for Embedded Systems Networking. *IEEE 24th International Conference on Software Quality, Reliability and Security (QRS)*, 250-261. Retrieved from https://doi.org/10.1109/QRS62785.2024.00033

Krebs, R., & Mazumdar, S. (2025). Analyzing LLM-Generated Code According to Four ISO/IEC 5055:2021 Categories. *IEEE Access*, 202482-202499. Retrieved from https://doi.org/10.1109/ACCESS.2025.3637569

Kumakale , S., A , A., Badiger , V. S., Sheetal , Jacintha , A., & Jagadish , S. (2026). Detection of Vulnerabilities in AI-Generated Java Code using Random Forest Algorithm. *International Conference on Intelligent and Innovative Technologies in Computing, Electrical and Electronics (IITCEE)*, 1-6. Retrieved from https://doi.org/10.1109/IITCEE67948.2026.11394305

Licorish, S. A., Bajpai, A., Arora, C., Wang, F., & Tantithamthavorn, K. (2025). Comparing Human and LLM Generated Code: The Jury is Still Out! *arXiv*. Retrieved from https://doi.org/10.48550/arXiv.2501.16857

Radanliev, P. (2025). Collaborative penetration testing suite for emerging generative AI algorithms. *Applied Intelligence*, 1-33. Retrieved from https://doi.org/10.1007/s10489-025-06908-1

Sheng, Z., Chen, Z., Gu, S., Huang, H., Gu, G., & Huang, J. (2025). LLMs in Software Security: A Survey of Vulnerability Detection Techniques and Insights. *ACM Computing Surveys*, 35. Retrieved from https://dl.acm.org/doi/10.1145/3769082

Tony, C., Díaz Ferreyra, N. E., Mutas, M., Dhif, S., & Scandariato, R. (2025). Prompting Techniques for Secure Code Generation: A Systematic

Investigation. *ACM Transactions on Software Engineering and Methodology*, 255:1-255:53. Retrieved from https://doi.org/10.1145/3722108

Umama, Danyaro, K., Nasser, M., Zakari, A., Abdullahi, S., Khanzada, A., . . . Shoaib, S. (2025). LLM-Based Code Generation: A Systematic Literature Review With Technical and Demographic Insights. *IEEE*, 194915-194939. Retrieved from https://ieeexplore.ieee.org/document/11242137

Wang, J., Luo, X., Cao, L., He, H., Huang, H., Xie, J., . . . Cai, Y. (2024). Is Your AI-Generated Code Really Safe? Evaluating Large Language Models on Secure Code Generation with CodeSecEval. *arXiv*. Retrieved from https://arxiv.org/abs/2407.02395